

An Introduction to the RISC-V Architecture

An Open ISA for Research and Industry

Giuseppe Tagliavini

Assistant professor with Tenure Track (RTT)
Department of Information Science and Engineering (DISI)
Alma Mater Studiorum – Università di Bologna
Email: giuseppe.tagliavini@unibo.it

PULP Platform

Open Source Hardware, the way it should be!



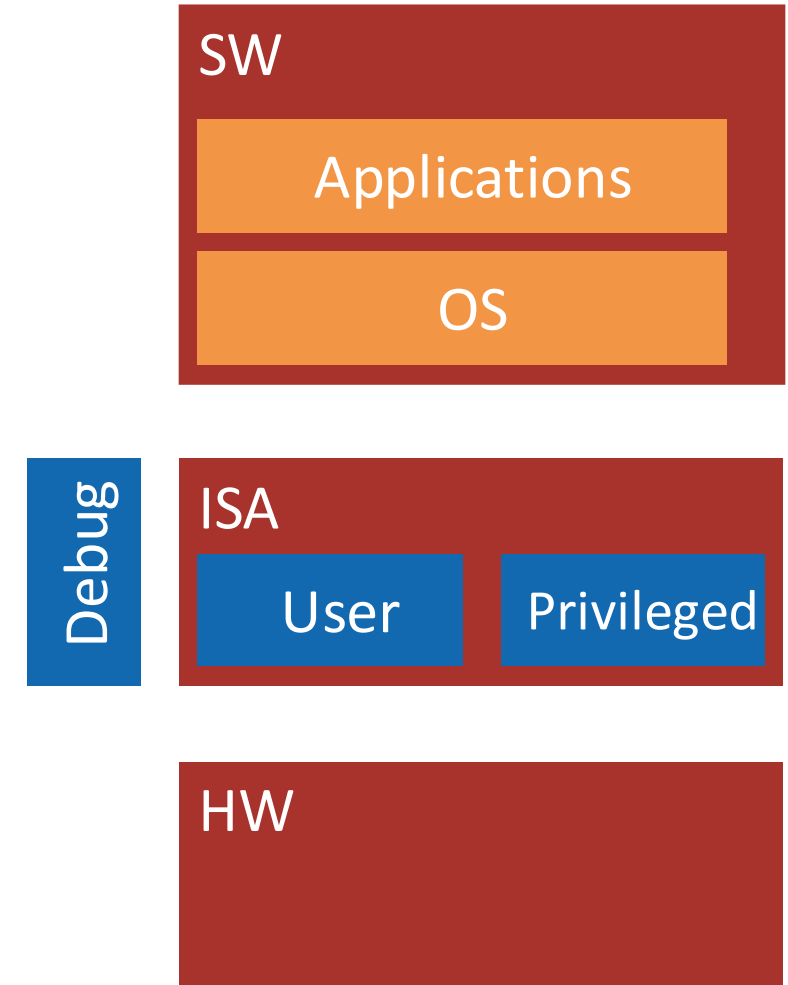
@pulp_platform 

pulp-platform.org 

youtube.com/pulp_platform 

RISC-V Instruction Set Architecture

- **Started by UC-Berkeley in 2010**
- **Contract between SW and HW**
 - Partitioned into *User* and *Privileged* specifications
 - External *Debug* interface
- **Standard governed by a foundation**
 - Necessary for the continuity
- **Specification for 32 / 64 / 128 bit ISA**
 - *No implementation, just the ISA*
 - Different implementations (both *open and close source*)



RISC-V International



RISC-V maintains a set of documents

[ABOUT](#)[SPECIFICATIONS](#)[DEVELOPERS](#)[INDUSTRIES](#)[COMMUNITY](#)[MEMBERS](#)[JOIN](#)

Ratified Specification

The RISC-V open-standard instruction set architecture (ISA) defines the fundamental guidelines for designing and implementing RISC-V processors.

[VIEW RATIFIED SPECS](#)[SPECS UNDER DEVELOPMENT](#)

RISC-V Software Ecosystem



- **Assembler:**
 - Binutils (upstream)
- **Compilers (godbolt.org)**
 - GCC (upstream)
 - LLVM (upstream)
- **Simulators:**
 - Spike (reference simulator)
 - QEMU, Gem5
- **Debugger: OpenOCD**
- **OS**
 - Linux, freeRTOS, Zephyr, seL4,
- **Hypervisors**
 - KVM, Xen, Bao, Jalihouse
- **Runtimes**
 - Jikes, Ocaml, Go

See <https://riscv.org/developers/#software>



RISC-V is modular by design



- The RISC-V ISA is **modular**, instructions are divided into **standard** and **non-standard extensions**
 - Standard extensions should be generally useful and ***should not conflict*** with other standard extensions
 - Non-standard extensions may be highly specialized, or ***may conflict*** with other standard or non-standard extensions
- The base integer ISA is named “I”, and contains *integer operations, integer loads, integer stores, and control-flow instructions*
 - This is mandatory for all RISC-V implementations
- An implementation is described with the notation **RV<word-width><extensions>**
 - E.g. RV32I = *32-bits architecture including only the base integer instruction set*



RISC-V standard extensions



- **M** → integer multiplication and division instructions
- **A** → instructions that atomically read, modify, and write memory
- **C** → compressed instructions (**16 bits** encoding)
- **F/D/Q** → Single/Double/Quad-precision floating point instructions
- **G** → placeholder to include the most common extensions (**IMAF[D]**)
- **C** → compressed instructions (**16 bits** encoding)
- ...



The FREEDOM in RISC-V is implementation

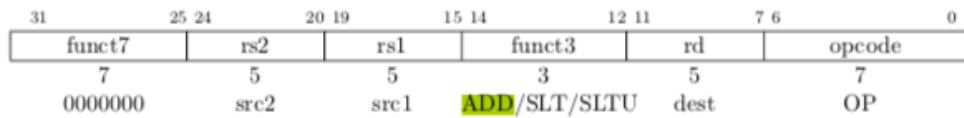


- In general, you can access all ISAs without restrictions
 - At most, SW tools need to be adapted to generate code for that ISA
- However, most ISAs are **closed**. Only specific vendors can implement it
 - To use a core that implements an ISA, you have to license/buy it from vendor
 - Open-source SW for the ISA is possible, but **building HW is not allowed**

RISC-V

Integer Register-Register Operations

RV32I defines several arithmetic R-type operations. All operations read the *rs1* and *rs2* registers as source operands and write the result into register *rd*. The *funct7* and *funct3* fields select the type of operation.



ARM

C2.9

ADD

Add without Carry.

Syntax

`ADD{S}{cond} {Rd}, Rn, Operand2`

`ADD{cond} {Rd}, Rn, #imm12 ; T32, 32-bit encoding only`



RISC-V is a load/store architecture

- **All operations are on internal registers**
 - Can not manipulate data in memory directly
- **Load instructions to copy from memory to registers**
- **Store instructions to copy from registers back to memory**
- **Branch and Jump instructions**



RISC-V Architectural State



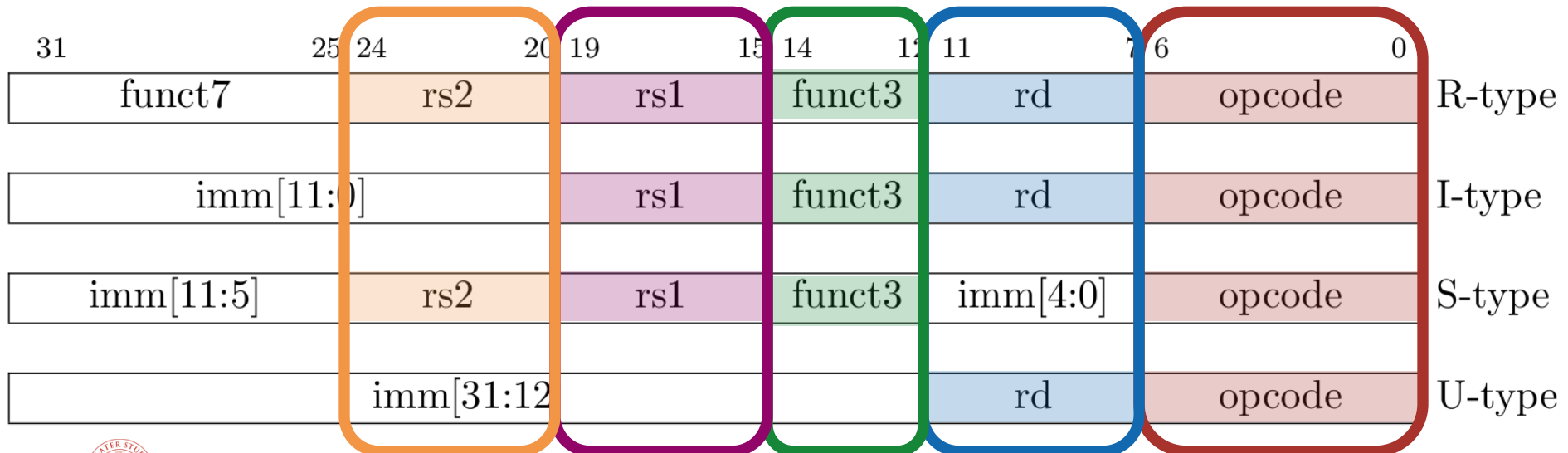
- **32 registers, each 32 / 64 / 128 bits long**
 - Named x0 to x31 (with semantic aliases)
 - x0 is hard wired to zero
 - There is a standard 'E' extension that uses only 16 registers (RV32E)
- **One program counter (PC)**
 - Byte based addressing, program counter increments by 4/8/16
- ***32 optional FP registers (for floating-point operation)***
- **Control Status Registers (CSRs)**
 - Encoding for up to 4'096 registers are reserved (not all are used)



Four basic instruction formats



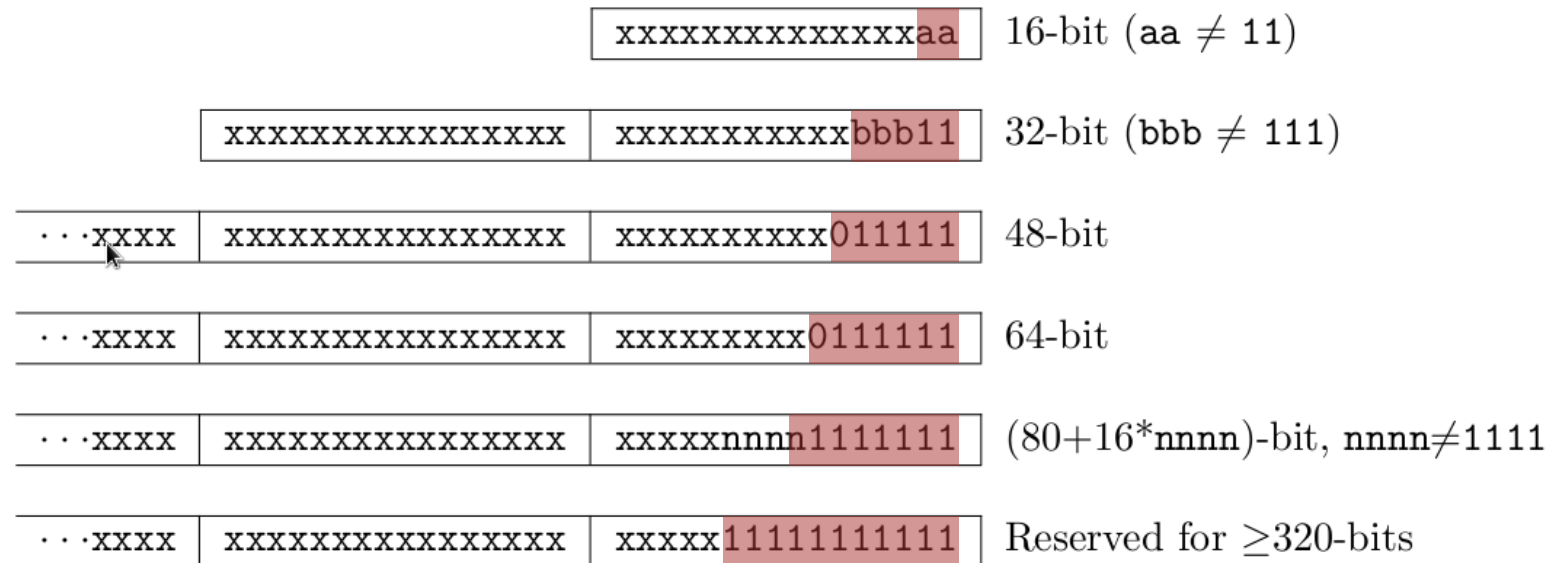
- **R** register to register operations
- **I** operations with immediate/constant values
- **S** operations with two source registers (B variant)
- **U** operations with large immediate/constant value (J variant)



RISC-V Instruction Length is Encoded



- **LSB of the instruction tells how long the instruction is**
- Supports **memory-aligned** instructions of **16, 32, 48, 64, 80, 96, ... , 320 bit**
 - Allows RISC-V to have Compressed instructions



Byte Address: base+4

base+2

base



Encoding of the instructions, main groups



- **Reserved** opcodes for standard extensions
- Rest of opcodes free for **custom** implementations
- Standard extensions will be frozen/not change in the future

inst[4:2]	000	001	010	011	100	101	110	111
inst[6:5]								(> 32b)
00	LOAD	LOAD-FP	custom-0	MISC-MEM	OP-IMM	AUIPC	OP-IMM-32	48b
01	STORE	STORE-FP	custom-1	AMO	OP	LUI	OP-32	64b
10	MADD	MSUB	NMSUB	NMADD	OP-FP	reserved	custom-2/rv128	48b
11	BRANCH	JALR	reserved	JAL	SYSTEM	reserved	custom-3/rv128	≥ 80b



R-format instructions



- R-format is used for **logical and arithmetic instructions**
- Same value for opcode, different encoding in **funct7#funct3**

0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
0000000	rs2	rs1	101	rd	0110011	SRL
0100000	rs2	rs1	101	rd	0110011	SRA
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND



I-format instructions



- I-format is used for **logical and arithmetic instructions with immediate**
- Shift by immediate instructions only use 5 bits → one of the higher order immediate bits is used to distinguish between logical and arithmetic shift

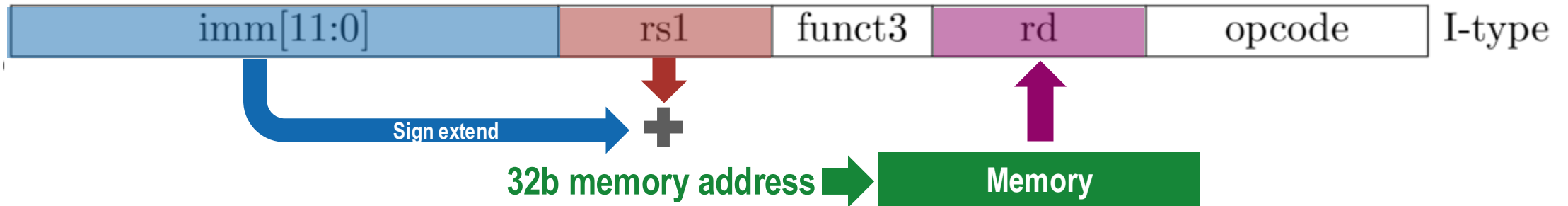
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI



Load instructions are also I-typed



ld x9, 64(x22)



- **Not possible to fit a 32b address in 32b encoding directly**
 - Take the content in source (**rs1**), add the immediate (**imm**) to it. This is the **address**
 - Read from this **address** in the memory and load into the destination (**rd**) register
- **RISC-V tries to minimize number of instructions**
 - The **ld** instruction seems overly complicated, but you can use this for everything



LOAD instructions



- The **funct3** field encodes both **size** and **signedness** of the data to load

imm[11:0]	rs1	000	rd	0000011	LB
imm[11:0]	rs1	001	rd	0000011	LH
imm[11:0]	rs1	010	rd	0000011	LW
imm[11:0]	rs1	100	rd	0000011	LBU
imm[11:0]	rs1	101	rd	0000011	LHU



S-format instructions



- Store instructions need to read **two registers** (rs1 for base memory address, rs2 for data to be stored) and an **immediate offset**
- **Why not using the I format?**
Again, to simplify the complexity of the decode stage

imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW



Branching instructions



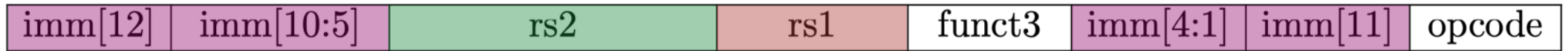
- Conditional branches: **beq, bne, bge ,blt**
 - specify an address to go to
 - take two registers to compare
 - no write into a register (similar to stores)
- Branches are typically used for control statements (*if-else, while, for*) and code areas are generally small (< 50 instructions)
- **PC-Relative Addressing:** Using the immediate field as a two's complement, we can specify $\pm 2^{11}$ addresses from the PC
 - Considering only even addressed, we can express $\pm 2^{12}$ values



Branching, how addresses come together



bne **x10**, **x0**, **L0** // if x10 != x11, jump to L0



- **Example:**

L0: lw x9, 0(x8)

add x11, x11, x9

addi x8, x8, 4

addi x10, x10, -1

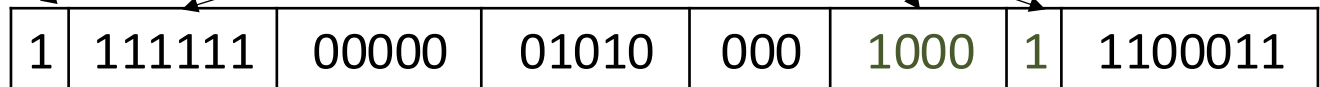
bne x10, x0, L0



-4 instructions
(-16 bytes)

13-bit immediate:

1111111110000 → discarded



B-format instructions



- **Why are the offset bits shuffled?**

Again, this design choice aims at simplifying the decode stage

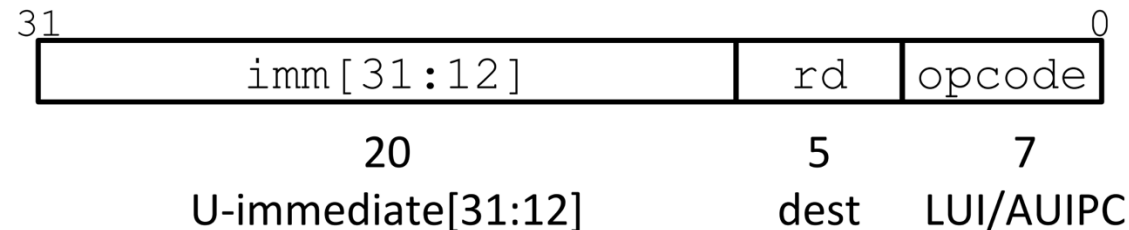
$\text{imm}[12 10:5]$	rs2	rs1	000	$\text{imm}[4:1 11]$	1100011	BEQ
$\text{imm}[12 10:5]$	rs2	rs1	001	$\text{imm}[4:1 11]$	1100011	BNE
$\text{imm}[12 10:5]$	rs2	rs1	100	$\text{imm}[4:1 11]$	1100011	BLT
$\text{imm}[12 10:5]$	rs2	rs1	101	$\text{imm}[4:1 11]$	1100011	BGE
$\text{imm}[12 10:5]$	rs2	rs1	110	$\text{imm}[4:1 11]$	1100011	BLTU
$\text{imm}[12 10:5]$	rs2	rs1	111	$\text{imm}[4:1 11]$	1100011	BGEU



U-format instructions



- The U format specifies a large (20 bits) immediate and a destination register
- This format is used by two instructions
 - **LUI** → Load Upper Immediate
 - **AUIPC** → Add Upper Immediate to PC



Creating long immediates with LUI



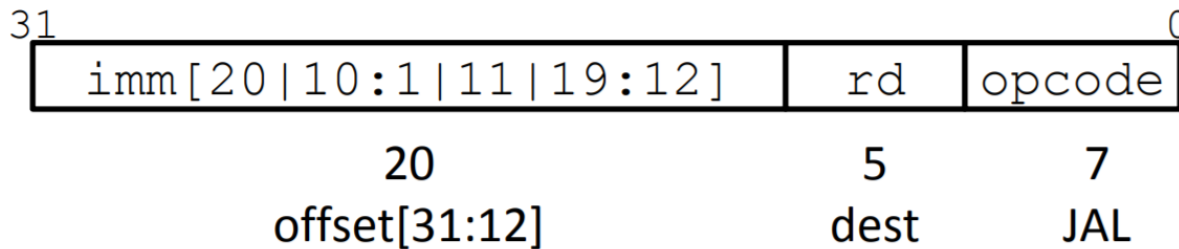
- **lui** writes the upper 20 bits of the destination with the immediate value, and clears the lower 12 bits
- Together with an **addi** to set low 12 bits, can create any 32-bit value in a register using two instructions (**lui/addi**)

```
lui    x10, 0x87654      # x10 = 0x87654000
addi   x10, x10, 0x321    # x10 = 0x87654321
```

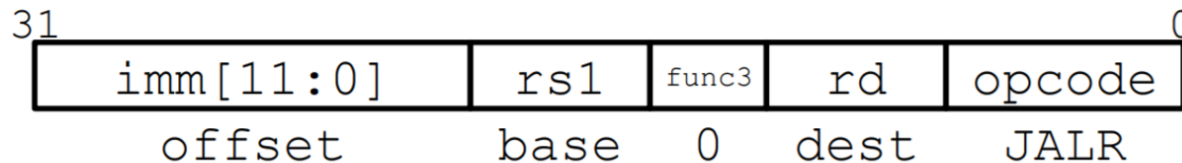


J-format instructions

- This format is dedicated to the JAL instruction
 - J is derived by JAL: `j Label` $\rightarrow$ `jal x0, Label`
 - JALR uses the I format



Saves PC+4 in register rd
Set PC = PC + offset



`jalr rd, rs1, offset`
Sets PC = rs1 + offset



Immediates in instructions: a final view



- Instruction types according to immediate encoding

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0	
funct7				rs2			rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type	
imm[11:5]			rs2			rs1		funct3		imm[4:0]		opcode		S-type	
imm[12]	imm[10:5]		rs2			rs1		funct3		imm[4:1]	imm[11]	opcode		B-type	
imm[31:12]										rd		opcode		U-type	
imm[20]	imm[10:1]				imm[11]		imm[19:12]			rd		opcode		J-type	



Compressed Instruction extension 'C' (1/2)



- Adoption of the C extension is a little bit in contrast with the decode simplification principle, however...

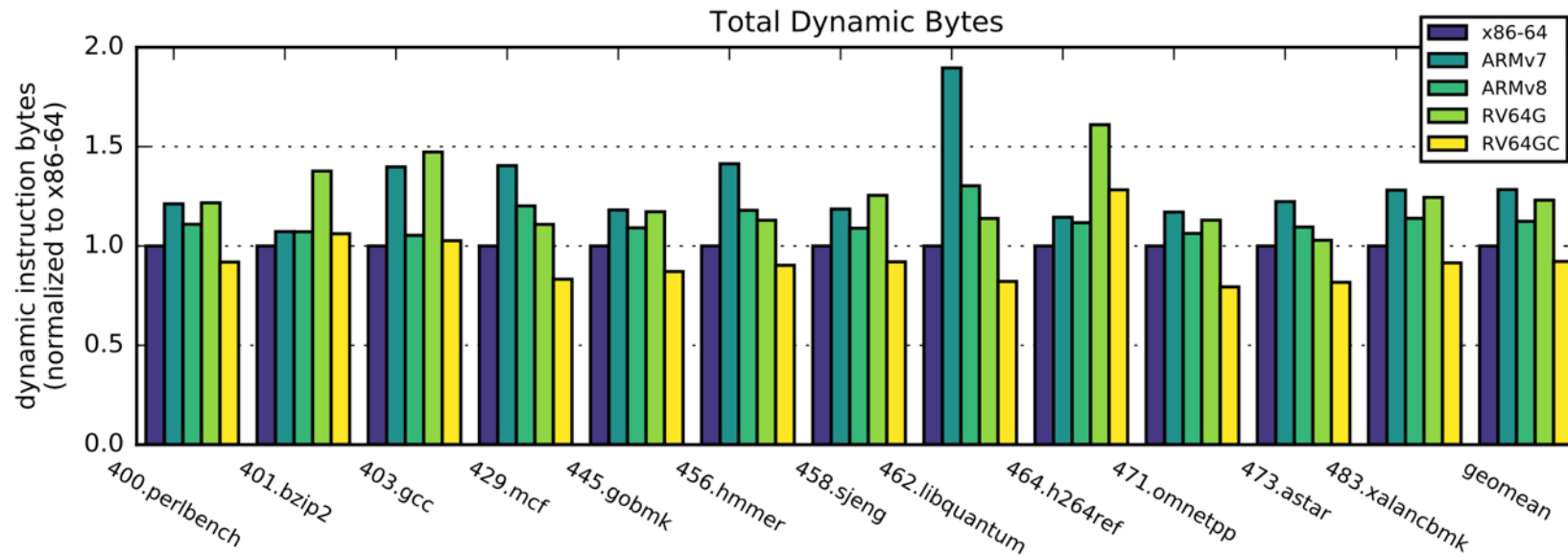
Format	Meaning	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
CR	Register	funct4				rd/rs1				rs2				op					
CI	Immediate	funct3		imm		rd/rs1				imm				op					
CSS	Stack-relative Store	funct3		imm						rs2				op					
CIW	Wide Immediate	funct3		imm								rd'		op					
CL	Load	funct3		imm			rs1'		imm		rd'		op						
CS	Store	funct3		imm			rs1'		imm		rs2'		op						
CB	Branch	funct3		offset			rs1'		offset				op						
CJ	Jump	funct3		jump target												op			



Compressed Instruction extension 'C' (2/2)



- ... there are advantages in adopting 16-bit instructions for common operations
 - Code size reduction by 34 %
 - Compressed instructions increase fetch-bandwidth
 - Allow for macro-op fusion of common patterns



x86-64: 3.71 bytes / instruction **RV64IC:** 3.00 bytes / instruction



Comparison with DLX



	DLX	RISC-V
Instruction Length	Always 32 bits	32 bits (base), variable with extensions
Formats	3 (R, I, J)	6 (R, I, S, B, U, J)
Design Purpose	Educational model	Scalable real-world architecture
Encoding	Minimal, fewer opcode bits	Richer encoding, extensible
Extensibility	Limited	Highly modular and extensible





Giuseppe Tagliavini (giuseppe.tagliavini@unibo.it)



<https://pulp-platform.org/>

